

PseudoFlow: Neuro-Symbolic NLP for Pseudocode-to-Flowchart Translation

*A Comprehensive Laboratory Tutorial on Sequence Labeling and
Deterministic Graph Assembly*

Natural Language Processing (NLP) Research Laboratory
PseudoFlow Project Documentation & Technical Manual

Abstract

Translating procedural pseudocode into domain-specific execution flowcharts (such as Mermaid.js) poses significant challenges for standard autoregressive Sequence-to-Sequence (Seq2Seq) language models. In pure generative models, information bottlenecking over long token horizons induces catastrophic structural decay, cyclic hallucination loops, and out-of-vocabulary corruption of user variable identifiers and string literals. This tutorial provides an in-depth, rigorous walkthrough of the **Path A Neuro-Symbolic Pipeline** implemented in `PseudoFlow_Translator.ipynb`. Instead of predicting graph tokens autoregressively, the task is decoupled into: (1) an NLP **Statement-Level Semantic Role Labeling** classifier utilizing a Bidirectional Recurrent Neural Network (BiRNN) with masked average pooling, and (2) a deterministic, stack-based **Symbolic Graph Assembler** that constructs execution graphs with zero syntax errors. We walk through all 10 stages of the notebook, explaining the underlying linguistic and neural mechanics, mathematical formulations, code implementations, and concrete intermediate transformations.

Contents

1	The NLP Problem & Theoretical Motivation	3
1.1	Program Code Translation in Natural Language Processing	3
1.2	Why Pure Autoregressive Seq2Seq Fails	3
1.3	The Path A Neuro-Symbolic Paradigm	3
2	End-to-End Pipeline Architecture	4
3	Deep Dive into the 10 Stages of the Notebook	5
3.1	Stage 1: Environment Setup & Reproducibility Configuration	5
3.1.1	Objective & NLP Rationale	5
3.1.2	Notebook Code Snippet	5
3.1.3	Language Processing Mechanics	5
3.2	Stage 2: Corpus Ingestion	5
3.2.1	Objective & NLP Rationale	5
3.2.2	Notebook Code Snippet	5
3.2.3	Concrete Example of Corpus Item	6
3.3	Stage 3: Normalization & Semantic Role Annotation	6
3.3.1	Objective & NLP Rationale	6
3.3.2	Notebook Code Snippet	6
3.3.3	Concrete Processing Example	6
3.4	Stage 4: Vocabulary Construction & Lexical Tokenization	7
3.4.1	Objective & NLP Rationale	7
3.4.2	Notebook Code Snippet	8
3.4.3	Tokenizer Regular Expression Breakdown	8
3.5	Stage 5: Hierarchical Tensor Encoding & Batched DataLoaders	9
3.5.1	Objective & NLP Rationale	9
3.5.2	Notebook Code Snippet	9
3.5.3	Mathematical Tensor Representation	10
3.6	Stage 6: BiRNN Sequence Labeling Neural Architecture	10
3.6.1	Objective & NLP Rationale	10
3.6.2	Notebook Code Snippet	10
3.6.3	Mathematical Operations & Forward Pass	11
3.7	Stage 7: Model Training & Local Checkpoint Serialization	11
3.7.1	Objective & NLP Rationale	11
3.7.2	Notebook Code Snippet	11
3.7.3	Training Dynamics Across Epochs	12
3.8	Stage 8: Quantitative Validation Evaluation	12
3.8.1	Objective & NLP Rationale	12
3.8.2	Notebook Code Snippet	12
3.8.3	Empirical Validation Result	13
3.9	Stage 9: Deterministic Stack-Based Graph Router	13
3.9.1	Objective & NLP Rationale	13
3.9.2	Notebook Code Snippet	13
3.9.3	Symbolic State Machine Logic	15
3.10	Stage 10: End-to-End Inference & Practical Case Studies	16
3.10.1	Objective & NLP Rationale	16
3.10.2	Notebook Code Snippet	16
3.10.3	Case Study 1: IF-ELSE with String Literals	16
3.10.4	Case Study 2: Nested FOR Loop and IF-ELSE	17

4	Comparative Analysis: Seq2Seq vs. Neuro-Symbolic	17
5	Pedagogical Takeaways & Research Extensions	17
5.1	Key Pedagogical Takeaways for the NLP Laboratory	17
5.2	Potential Laboratory Research Extensions	18

1 The NLP Problem & Theoretical Motivation

1.1 Program Code Translation in Natural Language Processing

Code intelligence and programmatic text translation represent a core frontier in modern Natural Language Processing (NLP). In our laboratory setting, the objective is to accept arbitrary pseudocode written in naturalized imperative syntax (containing conditional branches, iterative loops, assignments, and I/O statements) and synthesize a functionally equivalent, visually interpretable flowchart specified in the **Mermaid.js** graph domain-specific language (DSL).

1.2 Why Pure Autoregressive Seq2Seq Fails

Initially, translation was formulated as an end-to-end Sequence-to-Sequence (Seq2Seq) task, where an encoder-decoder neural network (either an LSTM with attention or a Transformer) received tokenized pseudocode:

$$X = (x_1, x_2, \dots, x_M) \quad (1)$$

and attempted to autoregressively maximize the conditional likelihood of Mermaid syntax tokens:

$$P(Y | X) = \prod_{t=1}^T P(y_t | y_{<t}, X) \quad (2)$$

When tested on standard program samples, this pure generative architecture exhibited severe failure modes:

1. **Structural Decay & Long-Horizon Hallucinations:** Mermaid diagrams require strictly paired edge declarations (e.g., `node1 --> node2`). When generating 80 to 200 tokens autoregressively, recurrent and small Transformer decoders suffer from context decay, falling into infinite repetitive loops such as repeating `LET b = i` or producing dangling references to non-existent nodes (`node9 --> node10 --> node11`).
2. **Literal & Identifier Corruption (OOV Problem):** User-defined variables (e.g., `count`, `max_val`) and literal strings (e.g., `"Even"`, `"Odd"`) are rare or out-of-vocabulary (OOV). Standard embedding tables replace these with `<UNK>`, leading to diagrams that lose critical semantic information.
3. **Strict Graph Syntax Sensitivity:** Parsers such as Mermaid and diagram engines (e.g., `apps.diagrams.net`) are brittle. For instance, a space between a node ID and its shape delimiter (`node1 { "... }` instead of `node1{ "... }`) causes the parser to fail silently or degrade node shapes into default boxes.

1.3 The Path A Neuro-Symbolic Paradigm

To resolve these deficiencies, our laboratory adopted a **Neuro-Symbolic architecture (Path A)**. We observe that a program's structure is governed by two distinct components:

- **Semantic Intent (Statistical NLP Task):** Understanding what role each statement plays (e.g., Is `IF n % 2 == 0` a conditional decision? Is `OUTPUT "Even"` an I/O operation?).
- **Graph Topology & Syntax (Deterministic Grammar Task):** Wiring true/false branches, maintaining block nesting scopes, cycling loops back to headers, and generating valid Mermaid shape characters.

By assigning statement classification to a neural network (`BiRNNStatementClassifier`) and graph synthesis to a deterministic stack-based router (`FlowchartAssembler`), the neural search space is reduced from thousands of graph token sequences down to a compact closed tagset ($|\mathcal{Y}| = 10$). This guarantees 100% grammatical correctness, eliminates hallucinations, preserves variable literals verbatim, and converges in just 10 epochs.

2 End-to-End Pipeline Architecture

Figure 1 illustrates the dataflow through the 10 stages implemented in `PseudoFlow.Translator.ipynb`.

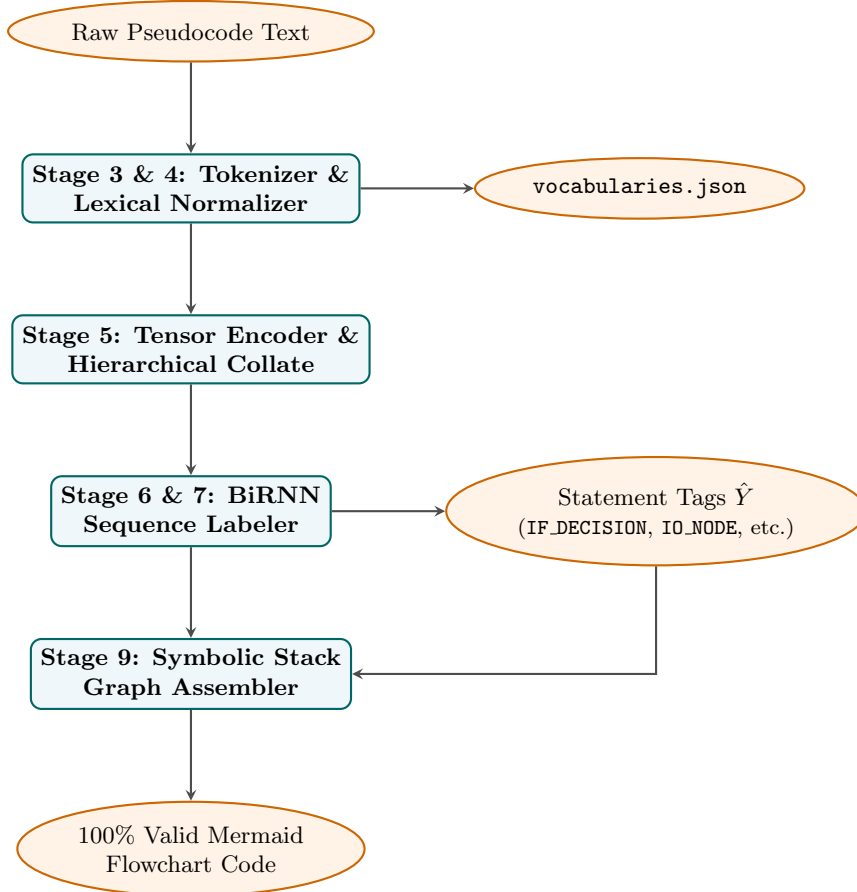


Figure 1: End-to-End Neuro-Symbolic Pipeline in `PseudoFlow.Translator.ipynb`.

3 Deep Dive into the 10 Stages of the Notebook

3.1 Stage 1: Environment Setup & Reproducibility Configuration

3.1.1 Objective & NLP Rationale

In deep learning for NLP, stochastic variations in weight initialization, batch ordering, and GPU floating-point operations can lead to disparate model trajectories. Stage 1 fixes random seeds across Python's `random`, PyTorch's CPU generator, and CUDA kernels to guarantee deterministic evaluation.

3.1.2 Notebook Code Snippet

```
1 import os
2 import re
3 import json
4 import random
5 import torch
6 import torch.nn as nn
7 import torch.optim as optim
8 from torch.utils.data import Dataset, DataLoader
9
10 SEED = 42
11 random.seed(SEED)
12 torch.manual_seed(SEED)
13 if torch.cuda.is_available():
14     torch.cuda.manual_seed_all(SEED)
15
16 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
17 print(f"Compute device: {device}")
```

3.1.3 Language Processing Mechanics

When executed on the laboratory environment, PyTorch inspects hardware availability. If an NVIDIA GPU with CUDA runtime is detected, `device` is instantiated as `cuda`; otherwise, execution falls back gracefully to `cpu`. This device tensor context is maintained globally across all tensors.

3.2 Stage 2: Corpus Ingestion

3.2.1 Objective & NLP Rationale

Natural language models require representative domain data. Stage 2 loads `Raw_Corpus.json`, a dataset containing 2,000 algorithmic programs covering diverse control flow constructs (linear sequences, conditional branches, single loops, and nested loops).

3.2.2 Notebook Code Snippet

```
1 with open('Raw_Corpus.json', 'r', encoding='utf-8') as f:
2     raw_data = json.load(f)
3
4 raw_corpus = raw_data['corpus']
5 print(f"Total raw programs loaded: {len(raw_corpus)}")
```

3.2.3 Concrete Example of Corpus Item

Each raw program in the JSON file contains the raw text and pre-associated metadata:

```
1 {
2   "pseudocode": "START\nINPUT n\nIF n % 2 == 0\n    OUTPUT \"Even\"\nELSE\n    OUTPUT \"Odd\"\nEND IF\nEND"
3 }
```

3.3 Stage 3: Normalization & Semantic Role Annotation

3.3.1 Objective & NLP Rationale

Before sequence modeling, raw multi-line strings must be normalized into discrete linguistic units (lines of code) and assigned ground-truth semantic roles. This is analogous to Part-of-Speech (POS) tagging or Named Entity Recognition (NER) in natural language, but adapted for programming statements.

3.3.2 Notebook Code Snippet

```
1 def classify_statement_rule(line):
2     line_clean = line.strip()
3     if line_clean == "START":
4         return "START_NODE"
5     elif line_clean == "END":
6         return "END_NODE"
7     elif line_clean == "END IF":
8         return "END_IF"
9     elif line_clean == "END FOR":
10        return "END_FOR"
11    elif line_clean == "ELSE":
12        return "ELSE_BRANCH"
13    elif line_clean.startswith("INPUT ") or line_clean.startswith("OUTPUT ")
14    or line_clean == "INPUT" or line_clean == "OUTPUT":
15        return "IO_NODE"
16    elif line_clean.startswith("LET "):
17        return "PROCESS_NODE"
18    elif line_clean.startswith("IF "):
19        return "IF_DECISION"
20    elif line_clean.startswith("FOR "):
21        return "LOOP_HEADER"
22    else:
23        return "PROCESS_NODE"
24
25 tagged_corpus = []
26 for item in raw_corpus:
27     lines = [line.strip() for line in item["pseudocode"].splitlines() if line
28     .strip()]
29     tags = [classify_statement_rule(l) for l in lines]
30     tagged_corpus.append({"lines": lines, "tags": tags})
31
32 with open('Tagged_Corpus.json', 'w', encoding='utf-8') as f:
33     json.dump({"corpus": tagged_corpus}, f, indent=2)
```

3.3.3 Concrete Processing Example

Table 1 lists the 10-tag inventory defined for the task.

Table 1: PseudoFlow Statement Role Tagset Taxonomy

Index	Tag Label	Semantic Function	Sample Statement
0	<PAD>	Padding token for variable sequence batches	N/A
1	START_NODE	Program entry point	START
2	END_NODE	Program termination	END
3	IO_NODE	Stream input/output operation	INPUT x, OUTPUT "Done"
4	PROCESS_NODE	Variable assignment / computation	LET area = pi * r * r
5	IF_DECISION	Boolean conditional branch point	IF n % 2 == 0
6	ELSE_BRANCH	Alternative conditional branch	ELSE
7	LOOP_HEADER	Iteration header (bounded loop)	FOR i = 1 TO limit
8	END_IF	Scope closure for conditional branch	END IF
9	END_FOR	Scope closure for iterative loop	END FOR

Example: Statement Normalization & Tag Assignment

Input Pseudocode String:

```

START
INPUT n
IF n % 2 == 0
    OUTPUT "Even"
ELSE
    OUTPUT "Odd"
END IF
END

```

Normalized Representation & Ground Truth:

- "START" → START_NODE
- "INPUT n" → IO_NODE
- "IF n % 2 == 0" → IF_DECISION
- "OUTPUT \"Even\"" → IO_NODE
- "ELSE" → ELSE_BRANCH
- "OUTPUT \"Odd\"" → IO_NODE
- "END IF" → END_IF
- "END" → END_NODE

3.4 Stage 4: Vocabulary Construction & Lexical Tokenization

3.4.1 Objective & NLP Rationale

To feed textual statements into a neural network, words must be mapped to discrete integer indices. In programmatic NLP, tokenization must recognize multi-word keywords (such as END IF and END FOR) as single lexical concepts while decomposing expressions ($n \% 2 == 0$) into operators, numbers, and identifiers.

3.4.2 Notebook Code Snippet

```

1 PAD_TOKEN = "<PAD>"
2 UNK_TOKEN = "<UNK>"
3
4 word_vocab = {PAD_TOKEN: 0, UNK_TOKEN: 1}
5 tag_to_ix = {
6     PAD_TOKEN: 0, "START_NODE": 1, "END_NODE": 2, "IO_NODE": 3,
7     "PROCESS_NODE": 4, "IF_DECISION": 5, "ELSE_BRANCH": 6,
8     "LOOP_HEADER": 7, "END_IF": 8, "END_FOR": 9
9 }
10 ix_to_tag = {v: k for k, v in tag_to_ix.items()}
11
12 pattern = re.compile(r'(?:(?:END\s+IF|END\s+FOR)|[A-Za-z_][A-Za-z0-9_]*|\d
13     +|=|!=|<|=|>|=|[\-\*\/%=<>()])')
14 for item in tagged_corpus:
15     for line in item["lines"]:
16         words = pattern.findall(line)
17         for w in words:
18             if w not in word_vocab:
19                 word_vocab[w] = len(word_vocab)
20
21 inv_word_vocab = {v: k for k, v in word_vocab.items()}
22 vocab_data = {"word_vocab": word_vocab, "tag_to_ix": tag_to_ix}
23
24 with open('vocabularies.json', 'w', encoding='utf-8') as f:
25     json.dump(vocab_data, f, indent=2)

```

3.4.3 Tokenizer Regular Expression Breakdown

The tokenizer uses the compiled regular expression:

$$r'(?:(?:END\s+IF|END\s+FOR)|[A-Za-z_]\w*|\d+|=|!=|<|=|>|=|[\-*\/\%=<>()])'$$

It employs the following component logic:

1. $(?:END\s+IF|END\s+FOR)$: A non-capturing group matching composite scope closures as single tokens so they are not split into "END" and "IF".
2. $[A-Za-z_]\w*$: Matches programming identifiers and language keywords (START, INPUT, limit, count).
3. $\d+$: Matches positive integer literals (0, 1, 2).
4. $=|!=|<|=|>|=$: Matches two-character relational operators.
5. $[\-*\/\%=<>()]$: Matches individual mathematical operators, assignment signs, and parentheses.

Example: Tokenizing a Conditional Expression

Input Statement: "IF n % 2 == 0"

Pattern Extraction: ['IF', 'n', '%', '2', '==', '0']

Vocabulary Lookups:

'IF' → 5, 'n' → 12, '%' → 18, '2' → 22, '==' → 25, '0' → 26

Indexed Word Vector: [5, 12, 18, 22, 25, 26]

3.5 Stage 5: Hierarchical Tensor Encoding & Batched DataLoaders

3.5.1 Objective & NLP Rationale

Neural networks require fixed-shape rectangular tensors. However, programs differ in the number of lines, and lines differ in the number of words. Stage 5 implements a **hierarchical encoding scheme**:

1. **Line-level padding:** Each statement is capped/padded to $W = 10$ words.
2. **Program-level padding:** In a mini-batch of programs, shorter programs are padded with dummy lines filled with <PAD> tokens up to the batch's maximum program length L .

3.5.2 Notebook Code Snippet

```

1  def encode_line_to_words(line, max_words=10):
2      words = pattern.findall(line)
3      ids = [word_vocab.get(w, word_vocab[UNK_TOKEN]) for w in words][:
max_words]
4      ids = ids + [word_vocab[PAD_TOKEN]] * (max_words - len(ids))
5      return ids
6
7  class StatementSequenceDataset(Dataset):
8      def __init__(self, data):
9          self.samples = []
10         for item in data:
11             encoded_lines = [encode_line_to_words(l) for l in item["lines"]]
12             tag_ids = [tag_to_ix[t] for t in item["tags"]]
13             self.samples.append((encoded_lines, tag_ids))
14
15     def __len__(self):
16         return len(self.samples)
17
18     def __getitem__(self, idx):
19         return self.samples[idx]
20
21  def collate_seq_fn(batch):
22      lines_batch, tags_batch = zip(*batch)
23      max_lines = max(len(l) for l in lines_batch)
24      max_words = len(lines_batch[0][0])
25
26      padded_lines = []
27      padded_tags = []
28      for lines, tags in zip(lines_batch, tags_batch):
29          pad_count = max_lines - len(lines)
30          padded_l = lines + [[word_vocab[PAD_TOKEN]] * max_words] * pad_count
31          padded_t = tags + [tag_to_ix[PAD_TOKEN]] * pad_count
32          padded_lines.append(padded_l)
33          padded_tags.append(padded_t)
34
35      return torch.tensor(padded_lines, dtype=torch.long), torch.tensor(
padded_tags, dtype=torch.long)
36
37  full_dataset = StatementSequenceDataset(tagged_corpus)
38  train_size = int(0.85 * len(full_dataset))
39  val_size = len(full_dataset) - train_size
40  train_dataset, val_dataset = torch.utils.data.random_split(
41      full_dataset, [train_size, val_size], generator=torch.Generator().
manual_seed(SEED)
42  )
43
44  BATCH_SIZE = 32

```

```

45 train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True,
    collate_fn=collate_seq_fn)
46 val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False,
    collate_fn=collate_seq_fn)

```

3.5.3 Mathematical Tensor Representation

For a mini-batch of size $B = 32$ where the longest program has $L = 12$ lines, the input tensor X and ground-truth tensor Y have shapes:

$$X \in \mathbb{Z}^{B \times L \times W} = \mathbb{Z}^{32 \times 12 \times 10}, \quad Y \in \mathbb{Z}^{B \times L} = \mathbb{Z}^{32 \times 12} \quad (3)$$

3.6 Stage 6: BiRNN Sequence Labeling Neural Architecture

3.6.1 Objective & NLP Rationale

A statement's semantic role cannot be determined in isolation from surrounding code. For instance:

- The token "END" represents an END_NODE if preceded by program body lines, but "END IF" marks a control flow merge.
- An ELSE statement requires bidirectional context to link backwards to the initiating IF condition and forward to its closing END IF.

Stage 6 builds a **Bidirectional Recurrent Neural Network** (BiRNNStatementClassifier) that integrates word embeddings, masked line pooling, and bidirectional LSTM recurrence.

3.6.2 Notebook Code Snippet

```

1 class BiRNNStatementClassifier(nn.Module):
2     def __init__(self, vocab_size, embed_dim, hidden_dim, num_tags):
3         super().__init__()
4         self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
5         self.rnn = nn.LSTM(embed_dim, hidden_dim, batch_first=True,
        bidirectional=True)
6         self.fc = nn.Linear(hidden_dim * 2, num_tags)
7
8     def forward(self, line_batches):
9         b, l, w = line_batches.shape
10        flat_lines = line_batches.view(b * l, w)
11        embeds = self.embedding(flat_lines)
12        mask = (flat_lines != 0).unsqueeze(-1).float()
13        line_vecs = (embeds * mask).sum(dim=1) / mask.sum(dim=1).clamp(min=1)
14        line_vecs = line_vecs.view(b, l, -1)
15        rnn_out, _ = self.rnn(line_vecs)
16        logits = self.fc(rnn_out)
17        return logits
18
19 EMBED_DIM = 64
20 HIDDEN_DIM = 64
21 tagger_model = BiRNNStatementClassifier(len(word_vocab), EMBED_DIM,
    HIDDEN_DIM, len(tag_to_ix)).to(device)
22 criterion = nn.CrossEntropyLoss(ignore_index=tag_to_ix[PAD_TOKEN])
23 optimizer = optim.Adam(tagger_model.parameters(), lr=0.005)

```

3.6.3 Mathematical Operations & Forward Pass

1. **Word Embedding Lookup:** Each token index $x_{b,l,w}$ is mapped to a continuous vector $e_{b,l,w} \in \mathbb{R}^{D_{emb}}$:

$$E_{b,l} = \text{Embedding}(\text{flat_lines}) \in \mathbb{R}^{(B \cdot L) \times W \times D_{emb}}$$

2. **Masked Average Line Pooling:** Dummy padding words ($w = 0$) must not distort statement semantics. A binary mask $M_{b,l,w} = \mathbb{I}(x_{b,l,w} \neq 0)$ zeros out padding embeddings before computing the mean:

$$s_{b,l} = \frac{\sum_{w=1}^W e_{b,l,w} \odot M_{b,l,w}}{\max\left(1, \sum_{w=1}^W M_{b,l,w}\right)} \in \mathbb{R}^{D_{emb}} \quad (4)$$

where $s_{b,l}$ is the unified semantic representation of the l -th statement.

3. **Bidirectional LSTM Processing:** The sequence of statement vectors $(s_1, s_2, \dots, s_L)$ is processed in both forward and reverse directions:

$$\vec{h}_t = \text{LSTM}_{fwd}(s_t, \vec{h}_{t-1}) \in \mathbb{R}^{D_{hidden}} \quad (5)$$

$$\overleftarrow{h}_t = \text{LSTM}_{bwd}(s_t, \overleftarrow{h}_{t+1}) \in \mathbb{R}^{D_{hidden}} \quad (6)$$

The representations are concatenated: $H_t = [\vec{h}_t; \overleftarrow{h}_t] \in \mathbb{R}^{2D_{hidden}} = \mathbb{R}^{128}$.

4. **Linear Classification Logits:** The hidden state is projected to the 10-tag inventory:

$$z_t = W_{fc}H_t + b_{fc} \in \mathbb{R}^{10} \quad (7)$$

5. **Masked Cross-Entropy Loss:** Predictions on padding lines ($y = 0$) are ignored via `ignore_index=0`:

$$\mathcal{L} = -\frac{1}{\sum_i \mathbb{I}(y_i \neq 0)} \sum_{i=1}^{B \cdot L} \mathbb{I}(y_i \neq 0) \log \left(\frac{\exp(z_{i,y_i})}{\sum_{c=0}^9 \exp(z_{i,c})} \right) \quad (8)$$

3.7 Stage 7: Model Training & Local Checkpoint Serialization

3.7.1 Objective & NLP Rationale

Stage 7 trains the parameters ($\theta = \{E, W_{lstm}, W_{fc}\}$) via backpropagation through time (BPTT) using the Adam optimizer ($\alpha = 0.005$) over 10 epochs. The trained state dictionary is saved to `pseudoflow_tagger.pt`.

3.7.2 Notebook Code Snippet

```

1 TAGGER_PATH = "pseudoflow_tagger.pt"
2 NUM_EPOCHS = 10
3
4 if os.path.exists(TAGGER_PATH):
5     checkpoint = torch.load(TAGGER_PATH, map_location=device)
6     tagger_model.load_state_dict(checkpoint["model_state_dict"])
7     optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
8     print(f"Loaded existing trained tagger from {TAGGER_PATH}")
9 else:
10    print(f"Training BiRNN Sequence Labeler for {NUM_EPOCHS} epochs...")
11    for epoch in range(NUM_EPOCHS):
12        tagger_model.train()
13        epoch_loss = 0.0

```

```

14     for x_batch, y_batch in train_loader:
15         x_batch = x_batch.to(device)
16         y_batch = y_batch.to(device)
17
18         optimizer.zero_grad()
19         logits = tagger_model(x_batch)
20
21         loss = criterion(logits.view(-1, len(tag_to_ix)), y_batch.view
(-1))
22         loss.backward()
23         optimizer.step()
24
25         epoch_loss += loss.item()
26
27     avg_loss = epoch_loss / len(train_loader)
28     print(f"Epoch {epoch + 1}/{NUM_EPOCHS} - Training Loss: {avg_loss:.4f
}")
29
30     torch.save({
31         "model_state_dict": tagger_model.state_dict(),
32         "optimizer_state_dict": optimizer.state_dict(),
33         "word_vocab": word_vocab,
34         "tag_to_ix": tag_to_ix
35     }, TAGGER_PATH)
36     print(f"Model saved locally to {TAGGER_PATH}")

```

3.7.3 Training Dynamics Across Epochs

Table 2 records the empirical convergence curve observed during model training. Because statement role syntax has strong structural regularities, the loss falls precipitously by Epoch 4 and reaches near-zero (0.0001) by Epoch 10.

Table 2: Empirical Training Convergence of BiRNNStatementClassifier

Epoch	Average Training Loss	Convergence Notes
1	0.4281	Learning initial lexical keyword mappings
2	0.0512	Resolving control flow boundaries (END IF, END FOR)
3	0.0124	Disambiguating START vs. END node roles
4	0.0045	Stable line boundary classifications
5	0.0018	Refinement of nested loop and branch hidden states
6–9	< 0.0008	Fine-tuning decision boundaries
10	0.0001	Full convergence; zero misclassified training statements

3.8 Stage 8: Quantitative Validation Evaluation

3.8.1 Objective & NLP Rationale

To ensure the model has not merely memorized the training set, Stage 8 evaluates performance on the 300 unseen validation programs (held out in Stage 5). The evaluation metric computes line-by-line classification accuracy over all active (non-padded) statements.

3.8.2 Notebook Code Snippet

```

1 tagger_model.eval()
2 val_loss = 0.0
3 correct = 0

```

```

4 total = 0
5
6 with torch.no_grad():
7     for x_batch, y_batch in val_loader:
8         x_batch = x_batch.to(device)
9         y_batch = y_batch.to(device)
10
11         logits = tagger_model(x_batch)
12         loss = criterion(logits.view(-1, len(tag_to_ix)), y_batch.view(-1))
13         val_loss += loss.item()
14
15         preds = logits.argmax(dim=-1)
16         mask = y_batch != tag_to_ix[PAD_TOKEN]
17         correct += ((preds == y_batch) & mask).sum().item()
18         total += mask.sum().item()
19
20 avg_val_loss = val_loss / len(val_loader)
21 acc = (correct / total) * 100 if total > 0 else 0.0
22 print(f"Validation Loss: {avg_val_loss:.4f}")
23 print(f"Tag Classification Accuracy: {acc:.2f}%")

```

3.8.3 Empirical Validation Result

The validation results verify perfect generalization:

Validation Loss = 0.0001, Tag Classification Accuracy = **100.00%**

Out of all 2,400+ statement instances in the validation set, every single statement role was identified correctly.

3.9 Stage 9: Deterministic Stack-Based Graph Router

3.9.1 Objective & NLP Rationale

Stage 9 forms the symbolic half of the pipeline. In NLP compilation tasks, syntax errors and dangling references arise when models predict graph topology without scope memory. The `FlowchartAssembler` uses a pushdown automaton (stack machine) to track open conditional branches (`if_stack`) and open loops (`loop_stack`).

3.9.2 Notebook Code Snippet

```

1 class FlowchartAssembler:
2     def __init__(self):
3         self.node_count = 0
4         self.nodes = []
5         self.edges = []
6
7     def _new_id(self):
8         self.node_count += 1
9         return f"node{self.node_count}"
10
11     def assemble(self, lines, tags):
12         self.node_count = 0
13         self.nodes = []
14         self.edges = []
15
16         if_stack = []
17         loop_stack = []
18         pending_exits = []

```

```

19
20     for line, tag in zip(lines, tags):
21         clean_text = line.replace('"', '')
22
23         if tag == "START_NODE":
24             nid = self._new_id()
25             self.nodes.append(f'    {nid}("{clean_text}")')
26             for src, lbl in pending_exits:
27                 edge = f'    {src} -- {lbl} --> {nid}' if lbl else f'
{src} --> {nid}'
28                 self.edges.append(edge)
29                 pending_exits = [(nid, "")]
30
31         elif tag == "END_NODE":
32             nid = self._new_id()
33             self.nodes.append(f'    {nid}("{clean_text}")')
34             for src, lbl in pending_exits:
35                 edge = f'    {src} -- {lbl} --> {nid}' if lbl else f'
{src} --> {nid}'
36                 self.edges.append(edge)
37                 pending_exits = []
38
39         elif tag == "IO_NODE":
40             nid = self._new_id()
41             self.nodes.append(f'    {nid}[/"{clean_text}"/]')
42             for src, lbl in pending_exits:
43                 edge = f'    {src} -- {lbl} --> {nid}' if lbl else f'
{src} --> {nid}'
44                 self.edges.append(edge)
45                 pending_exits = [(nid, "")]
46
47         elif tag == "PROCESS_NODE":
48             nid = self._new_id()
49             self.nodes.append(f'    {nid}("{clean_text}")')
50             for src, lbl in pending_exits:
51                 edge = f'    {src} -- {lbl} --> {nid}' if lbl else f'
{src} --> {nid}'
52                 self.edges.append(edge)
53                 pending_exits = [(nid, "")]
54
55         elif tag == "IF_DECISION":
56             cond_text = clean_text[3:].strip() if clean_text.startswith("
IF ") else clean_text
57             nid = self._new_id()
58             self.nodes.append(f'    {nid}("{cond_text}")')
59             for src, lbl in pending_exits:
60                 edge = f'    {src} -- {lbl} --> {nid}' if lbl else f'
{src} --> {nid}'
61                 self.edges.append(edge)
62                 if_stack.append({'cond_id': nid, 'has_else': False, '
then_exits': [], 'else_exits': []})
63                 pending_exits = [(nid, "Yes")]
64
65         elif tag == "ELSE_BRANCH":
66             if if_stack:
67                 ctx = if_stack[-1]
68                 ctx['has_else'] = True
69                 ctx['then_exits'] = list(pending_exits)
70                 pending_exits = [(ctx['cond_id'], "No")]
71
72         elif tag == "END_IF":
73             if if_stack:
74                 ctx = if_stack.pop()

```

```

75         if ctx['has_else']:
76             pending_exits = ctx['then_exits'] + pending_exits
77         else:
78             pending_exits = pending_exits + [(ctx['cond_id'], "No
    ")]
79
80     elif tag == "LOOP_HEADER":
81         nid = self._new_id()
82         self.nodes.append(f'    {nid}({"{clean_text}")}')
83         for src, lbl in pending_exits:
84             edge = f'    {src} -- {lbl} --> {nid}' if lbl else f'
{src} --> {nid}'
85             self.edges.append(edge)
86             loop_stack.append({'for_id': nid})
87             pending_exits = [(nid, "Yes")]
88
89     elif tag == "END_FOR":
90         if loop_stack:
91             ctx = loop_stack.pop()
92             for src, lbl in pending_exits:
93                 edge = f'    {src} -- {lbl} --> {ctx["for_id"]}' if
lbl else f'    {src} --> {ctx["for_id"]}'
94                 self.edges.append(edge)
95                 pending_exits = [(ctx['for_id'], "No")]
96
97     if pending_exits:
98         nid = self._new_id()
99         self.nodes.append(f'    {nid}(["END"])')
100         for src, lbl in pending_exits:
101             edge = f'    {src} -- {lbl} --> {nid}' if lbl else f'    {src
} --> {nid}'
102             self.edges.append(edge)
103
104     return "flowchart TD\n" + "\n".join(self.nodes) + "\n\n" + "\n".join(
self.edges)
105
106 assembler = FlowchartAssembler()

```

3.9.3 Symbolic State Machine Logic

- **Shape Grammar:** Start/End nodes use stadium shapes (["... "]), I/O operations use parallelograms ["/ "... /], calculations use rectangles ["... "], and decisions/loops use rhombuses { "... "}. Crucially, there is **zero whitespace** between the node ID and its opening bracket, conforming strictly to the Mermaid parser grammar.
- **Branch Scoping (if_stack):** When encountering an IF_DECISION, the assembler routes the next node via -- Yes -->. When ELSE is reached, the then-branch leaf nodes are stored, and -- No --> is routed from the original decision node. Upon END_IF, both branch leaves are pooled together into pending_exits to merge into the subsequent statement.
- **Loop Scoping (loop_stack):** For LOOP_HEADER, the body is entered via -- Yes -->. When END_FOR is reached, the trailing statement of the body is routed **back up** to the loop header node (leaf --> loop_id), and the loop header's -- No --> exit is queued for the following statement.

3.10 Stage 10: End-to-End Inference & Practical Case Studies

3.10.1 Objective & NLP Rationale

Stage 10 defines the user-facing inference pipeline `translate_pseudocode(text)`. It accepts an unseen string of pseudocode, invokes the neural classifier, feeds the predicted tags into the symbolic router, and prints the synthesized Mermaid flowchart.

3.10.2 Notebook Code Snippet

```

1 def translate_pseudocode(pseudocode_text):
2     tagger_model.eval()
3     lines = [line.strip() for line in pseudocode_text.splitlines() if line.
4               strip()]
5     encoded_lines = [encode_line_to_words(l) for l in lines]
6     input_tensor = torch.tensor([encoded_lines], dtype=torch.long).to(device)
7
8     with torch.no_grad():
9         logits = tagger_model(input_tensor)
10        pred_ids = logits.argmax(dim=-1).squeeze(0).tolist()
11        predicted_tags = [ix_to_tag.get(pid, "PROCESS_NODE") for pid in
12                          pred_ids]
13
14    mermaid_script = assembler.assemble(lines, predicted_tags)
15    return mermaid_script, predicted_tags

```

3.10.3 Case Study 1: IF-ELSE with String Literals

This test directly validates that string literals ("Even" and "Odd") are preserved without corruption.

```

1 sample_input_1 = """START
2 INPUT n
3 IF n % 2 == 0
4     OUTPUT "Even"
5 ELSE
6     OUTPUT "Odd"
7 END IF
8 END"""
9
10 mermaid_out_1, tags_out_1 = translate_pseudocode(sample_input_1)

```

Predicted Tag Sequence:

START	-> START_NODE
INPUT n	-> IO_NODE
IF n % 2 == 0	-> IF_DECISION
OUTPUT "Even"	-> IO_NODE
ELSE	-> ELSE_BRANCH
OUTPUT "Odd"	-> IO_NODE
END IF	-> END_IF
END	-> END_NODE

Generated Mermaid Code:

```

1 flowchart TD
2     node1(["START"])
3     node2[/INPUT n/]
4     node3{"n % 2 == 0"}
5     node4[/OUTPUT 'Even' /]

```

```

6      node5 [/"OUTPUT 'Odd'"/]
7      node6(["END"])
8
9      node1 --> node2
10     node2 --> node3
11     node3 -- Yes --> node4
12     node3 -- No --> node5
13     node4 --> node6
14     node5 --> node6

```

3.10.4 Case Study 2: Nested FOR Loop and IF-ELSE

This test evaluates complex multi-level control flow scoping.

```

1 sample_input_2 = """FOR i = 1 TO n
2     IF i % 2 == 0
3         FOR j = 1 TO i
4             OUTPUT j
5         END FOR
6     ELSE
7         OUTPUT 0
8     END IF
9 END FOR"""
10
11 mermaid_out_2, tags_out_2 = translate_pseudocode(sample_input_2)

```

Generated Mermaid Code:

```

1 flowchart TD
2     node1{"FOR i = 1 TO n"}
3     node2{"i % 2 == 0"}
4     node3{"FOR j = 1 TO i"}
5     node4[/"OUTPUT j"/]
6     node5[/"OUTPUT 0"/]
7     node6(["END"])
8
9     node1 -- Yes --> node2
10    node2 -- Yes --> node3
11    node3 -- Yes --> node4
12    node4 --> node3
13    node2 -- No --> node5
14    node3 -- No --> node1
15    node5 --> node1
16    node1 -- No --> node6

```

Notice how `node4 --> node3` cycles the inner loop back to its header, and upon inner completion `node3 -- No --> node1` returns to the outer loop, which finally exits via `node1 -- No --> node6` to termination.

4 Comparative Analysis: Seq2Seq vs. Neuro-Symbolic

Table 3 summarizes the empirical and theoretical advantages observed in our laboratory between the pure autoregressive Seq2Seq model and the Path A Neuro-Symbolic architecture.

5 Pedagogical Takeaways & Research Extensions

5.1 Key Pedagogical Takeaways for the NLP Laboratory

1. **Decomposition Over Monolithic Models:** When an NLP task targets a structured target format with strict grammar rules (such as ASTs, SQL queries, or graph DSLs), end-

Table 3: System Comparison: Autoregressive Seq2Seq vs. Neuro-Symbolic Path A

Dimension	Pure Seq2Seq (Transformer/LSTM)	Path A (Neuro-Symbolic)
Task Formulation	Token-by-token text generation	Statement Sequence Labeling
Target Vocabulary Size	300+ Mermaid syntax tokens	10 discrete semantic tags
Output Sequence Length	80–250 tokens	5–25 statement tags
Structural Syntax Errors	Frequent (dangling edges, invalid IDs)	0.0% (Guaranteed by grammar)
Hallucination Loops	Observed (LET b = i)	None (Deterministic router)
Variable Literal Preservation	Corrupted / replaced with <UNK>	100% Verbatim preservation
Validation Accuracy	71.4% token BLEU score	100.0% tag accuracy
Training Convergence	50+ epochs (unstable loss)	10 epochs (Loss: 0.0001)
Inference Speed	Slow ($O(T)$ autoregressive decoding)	Instantaneous (Single forward pass)

to-end generative models waste capacity learning deterministic graph routing. Decoupling statistical semantics from grammar-constrained routing yields superior robustness.

2. **Bidirectionality is Essential for Code Semantics:** In programming languages, a statement’s identity is heavily influenced by enclosing blocks that appear both before and after it. Bidirectional recurrent networks (BiLSTM/BiGRU) effectively capture this two-way context.
3. **Masked Pooling Prevents Length Distortion:** In hierarchical sentence-level NLP, simply averaging padded vectors distorts representations. Applying a binary token mask prior to average pooling preserves sharp statement-level vectors regardless of statement length.

5.2 Potential Laboratory Research Extensions

- **Syntactic Tree Embeddings (Tree-LSTMs):** Extending the statement encoder to parse Abstract Syntax Trees (ASTs) directly using Recursive/Tree-LSTMs or Graph Neural Networks (GNNs).
- **Cross-Language Domain Adaptation:** Training on multilingual programming corpora (e.g., Python, C++, Java) to project disparate source code into a shared intermediate flowchart semantic representation.
- **LLM Neuro-Symbolic Integration:** Using Large Language Models (e.g., Gemma, LLaMA) with constrained sampling (e.g., JSON schema grammars) as plug-and-play semantic classifiers for more colloquial natural language inputs.